Nombre del estudiante: Eduardo Arturo Guzman Alvarez

Matrícula: 20240232

Materia: Programación paralela

Maestro: Erick Leonardo Perez Veloz

Título del proyecto:

Implementación Paralela de Algoritmos de Búsqueda en Grafos (BFS y DFS)

índice

1. Introducción .....
2. Descripción del Problema .....
3. Cumplimiento de los Requisitos del Proyecto .....
4. Diseño de la Solución .....
5. Implementación Técnica .....
6. Evaluación de Desempeño .....
7. Trabajo en Equipo .....
8. Conclusiones .....
9. Referencias .....
10. Anexos .....

1. INTRODUCCIÓN

1.1 Presentación general del proyecto

En la actualidad, las aplicaciones de movilidad como Uber y Google Maps dependen de algoritmos eficientes para calcular rutas óptimas entre ubicaciones en tiempo real. Estas soluciones se basan en la representación de redes de transporte como grafos, donde los nodos representan ubicaciones geográficas y las conexiones representan las vías disponibles.

El presente proyecto tiene como objetivo el diseño e implementación de un sistema de búsqueda de rutas utilizando algoritmos clásicos sobre grafos, específicamente **BFS (Breadth-First Search)** y **DFS (Depth-First Search)**, desarrollados tanto en versiones secuenciales como paralelas. La implementación se realiza en el lenguaje C#, utilizando la **Task Parallel Library (TPL)** para aprovechar la ejecución concurrente en múltiples núcleos del procesador.

Además, el sistema permite generar grafos de tamaño variable y evaluar el rendimiento de los algoritmos mediante métricas como el tiempo de ejecución, el speedup y la eficiencia, permitiendo analizar el impacto del paralelismo en la resolución del problema.

1.2 Justificación del tema elegido

El crecimiento de los sistemas computacionales modernos ha llevado a un aumento significativo en el uso de arquitecturas multinúcleo, lo que hace indispensable el uso de técnicas de programación paralela para mejorar el rendimiento de las aplicaciones.

En escenarios como los sistemas de navegación y transporte, el cálculo de rutas debe realizarse de manera rápida y eficiente, incluso cuando múltiples usuarios realizan solicitudes simultáneamente. En este contexto, el paralelismo no solo mejora los tiempos de respuesta, sino que también permite escalar el sistema para soportar grandes volúmenes de datos.

Se eligió este tema debido a su relevancia práctica y su capacidad para demostrar, de manera clara, cómo la paralelización de algoritmos tradicionales puede impactar significativamente en el desempeño de un sistema. Asimismo, permite analizar los retos asociados al uso de memoria compartida, sincronización y distribución de tareas.

1.3 Objetivos

1.3.1 Objetivo general

Desarrollar e implementar un sistema de búsqueda de rutas en grafos que permita comparar el rendimiento entre algoritmos secuenciales y paralelos, simulando un escenario similar al de aplicaciones de navegación como Uber o Google Maps.

1.3.2 Objetivos específicos

- Implementar los algoritmos BFS y DFS en su versión secuencial para la búsqueda de rutas en un grafo.
- Desarrollar versiones paralelas de dichos algoritmos utilizando múltiples hilos de ejecución.
- Evaluar el rendimiento de ambas versiones mediante métricas como tiempo de ejecución, speedup y eficiencia.
- Analizar la escalabilidad del sistema al variar la cantidad de núcleos utilizados.
- Identificar limitaciones y posibles cuellos de botella en la ejecución paralela.

2. Descripción del Problema

2.1 Contexto del problema seleccionado

En aplicaciones modernas de movilidad como Uber o Google Maps, uno de los principales desafíos es encontrar rutas eficientes entre dos puntos dentro de una red de transporte. Estas plataformas trabajan con grandes volúmenes de datos representados como grafos, donde los nodos corresponden a ubicaciones geográficas (intersecciones, direcciones o puntos de interés) y las aristas representan las conexiones entre ellas (calles o avenidas).

El problema fundamental consiste en determinar una ruta óptima entre un origen y un destino. Este tipo de problema se vuelve computacionalmente costoso cuando el número de nodos y conexiones crece significativamente, como ocurre en ciudades grandes.

En este proyecto, se modela este escenario mediante la generación de un grafo que simula una red de rutas, sobre la cual se aplican algoritmos de búsqueda como BFS (Breadth-First Search) y DFS (Depth-First Search) para encontrar caminos entre dos puntos específicos.

2.2 Aplicación del problema en un escenario real

En plataformas como Uber o Google Maps, el cálculo de rutas no solo se realiza una vez, sino múltiples veces de manera simultánea para distintos usuarios. Por ejemplo:

- Un conductor busca la mejor ruta hacia un pasajero.
- Un usuario solicita la ruta más rápida hacia su destino.
- El sistema recalcula rutas en tiempo real debido al tráfico.

Esto implica que el sistema debe procesar múltiples consultas concurrentes sobre estructuras de datos complejas. En este contexto, el uso de algoritmos de búsqueda eficientes es esencial para garantizar tiempos de respuesta bajos.

El proyecto desarrollado simula este comportamiento mediante la ejecución de búsquedas sobre un grafo, representando múltiples posibles rutas entre ubicaciones. Aunque el grafo es generado de forma aleatoria, este actúa como una aproximación válida de una red de transporte real, permitiendo evaluar el comportamiento de los algoritmos en diferentes condiciones.

2.3 Importancia del paralelismo en la solución

Dado que las aplicaciones de movilidad manejan millones de usuarios de forma simultánea, el uso de programación paralela es fundamental para mejorar el rendimiento del sistema.

El paralelismo permite:

- Ejecutar múltiples exploraciones del grafo al mismo tiempo.
- Reducir el tiempo de respuesta en la búsqueda de rutas.
- Aprovechar los múltiples núcleos del procesador.
- Escalar el sistema a medida que aumenta la carga de trabajo.

En este proyecto, se implementa una versión paralela del algoritmo BFS utilizando la biblioteca Task Parallel Library (TPL) de C#. Esta versión permite explorar diferentes nodos del grafo de manera concurrente, simulando el comportamiento de sistemas reales que procesan múltiples rutas simultáneamente.

Además, se analizan métricas como speedup y eficiencia para evaluar el impacto del paralelismo, evidenciando cómo el rendimiento mejora (o se ve afectado) al variar la cantidad de núcleos utilizados.

3. Cumplimiento de los Requisitos del Proyecto

3.1 Para cada criterio, se debe justificar cómo el proyecto lo aborda:

3.1.1 Ejecución simultánea de múltiples tareas

El proyecto implementa ejecución simultánea mediante el uso de la biblioteca **Task Parallel Library (TPL)** de C#, específicamente utilizando la estructura `Parallel.ForEach`.

En el algoritmo **BFS paralelo**, los nodos de un mismo nivel del grafo se procesan de manera concurrente. Cada hilo se encarga de explorar los vecinos de un nodo distinto, permitiendo que múltiples caminos potenciales sean evaluados al mismo tiempo.

De igual forma, en la implementación de **DFS paralelo**, se distribuyen los nodos vecinos iniciales entre varios hilos, permitiendo explorar diferentes ramas del grafo de manera simultánea.

Esto garantiza que el sistema aproveche los múltiples núcleos del procesador para reducir el tiempo total de ejecución.

3.1.2 Necesidad de compartir datos entre tareas

La ejecución paralela requiere compartir información entre los distintos hilos, especialmente para evitar la exploración redundante de nodos.

Para ello, se utilizan estructuras de datos concurrentes como:

- `ConcurrentDictionary`: utilizado para almacenar los nodos visitados y sus relaciones (por ejemplo, el nodo padre en BFS).
- `ConcurrentBag`: utilizado para almacenar los nodos del siguiente nivel en el BFS paralelo.

Estas estructuras permiten el acceso seguro desde múltiples hilos sin necesidad de implementar bloqueos manuales, evitando condiciones de carrera y garantizando la consistencia de los datos compartidos.

3.1.3 Exploración de diferentes estrategias de paralelización

El proyecto no se limita a una sola estrategia de paralelización, sino que implementa diferentes enfoques según el algoritmo:

- En **BFS paralelo**, se utiliza una estrategia por niveles (level-synchronous), donde todos los nodos de un nivel se procesan en paralelo antes de avanzar al siguiente.
- En **DFS paralelo**, se adopta una estrategia basada en la exploración concurrente de ramas independientes del grafo, distribuyendo el trabajo entre múltiples hilos desde el inicio.

Estas dos estrategias permiten analizar cómo diferentes enfoques de paralelización afectan el rendimiento y comportamiento del sistema.

3.1.4 Escalabilidad con más recursos

El sistema permite configurar explícitamente el número de núcleos utilizados en la ejecución paralela mediante el parámetro `MaxDegreeOfParallelism`.

Además, se realizan pruebas de escalabilidad ejecutando el algoritmo con diferentes cantidades de núcleos (por ejemplo: 1, 2, 4, 8), lo que permite observar cómo varía el rendimiento en función de los recursos disponibles.

Esto demuestra que la solución puede adaptarse a diferentes entornos de hardware y analizar su comportamiento a medida que se incrementa la capacidad de procesamiento.

3.1.5 Métricas de evaluación del rendimiento

El proyecto incluye la medición de métricas clave para evaluar el desempeño de la solución:

- **Tiempo de ejecución:** medido utilizando la clase `Stopwatch` de C#.
- **Speedup:** calculado como la relación entre el tiempo secuencial y el tiempo paralelo.
- **Eficiencia:** calculada en función del speedup y la cantidad de núcleos utilizados.

Estas métricas se calculan tanto para la ejecución con máximo paralelismo como para diferentes configuraciones de núcleos, permitiendo un análisis detallado del rendimiento del sistema.

3.1.6 Aplicación a un problema del mundo real

El proyecto se contextualiza en un escenario similar al de aplicaciones de movilidad como Uber o Google Maps, donde es necesario encontrar rutas entre ubicaciones dentro de una red de transporte.

El grafo generado representa una abstracción de dicha red, donde:

- Los nodos representan ubicaciones (intersecciones o puntos geográficos).
- Las conexiones representan posibles rutas entre dichas ubicaciones.

Aunque el grafo es generado de manera aleatoria, este permite simular múltiples escenarios y evaluar el comportamiento de los algoritmos bajo distintas condiciones, lo cual es una práctica común en el análisis de algoritmos antes de su implementación en sistemas reales.

El uso de paralelismo en este contexto resulta fundamental para mejorar los tiempos de respuesta y permitir la escalabilidad del sistema ante múltiples solicitudes simultáneas.

4. Diseño de la Solución

4.1 Arquitectura general del sistema

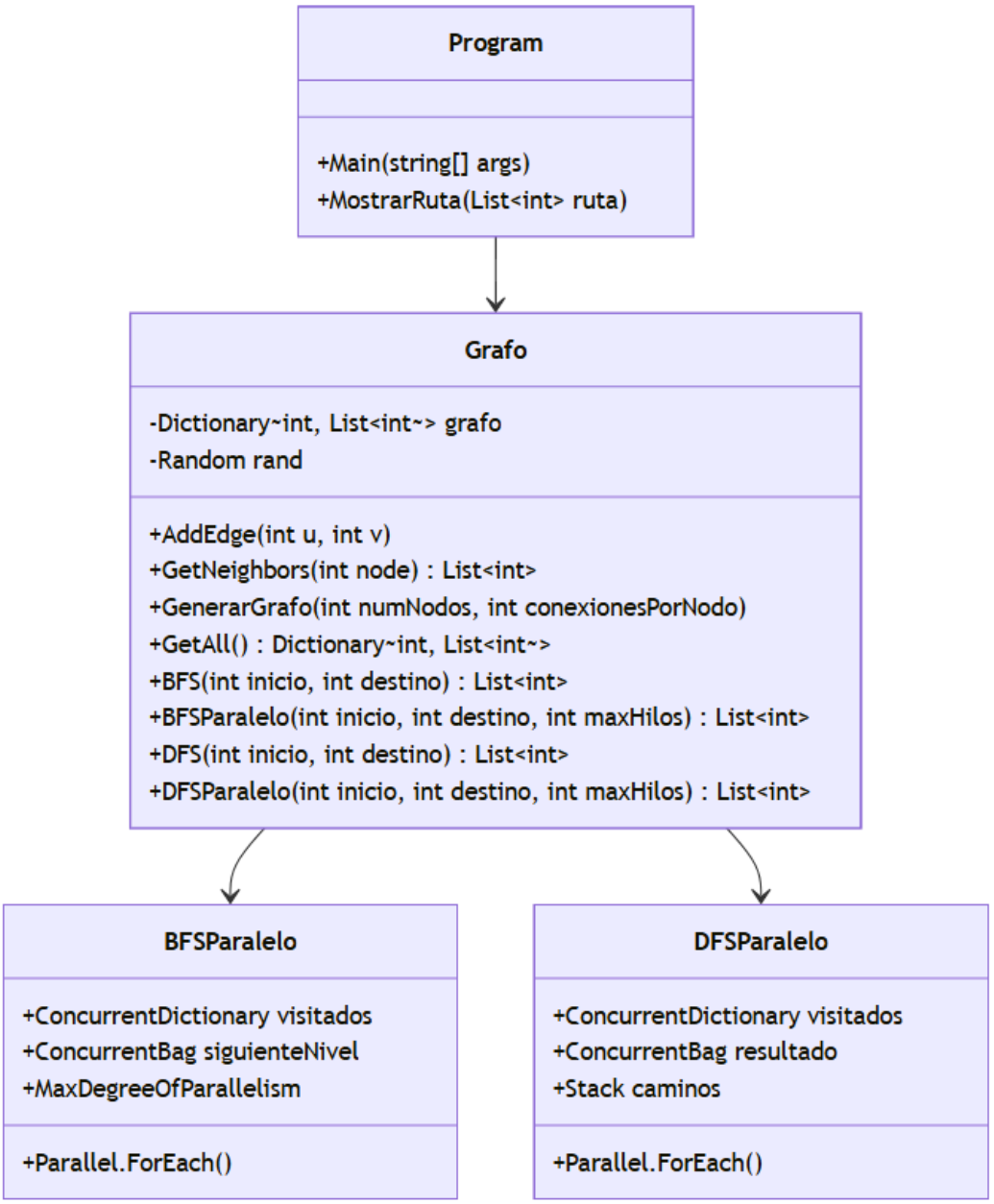
El sistema desarrollado sigue una arquitectura modular simple, compuesta por dos componentes principales:

1. **Clase Program (punto de entrada):**
    - Se encarga de la interacción con el usuario.
    - Solicita los parámetros de entrada (número de nodos y nodo destino).
    - Ejecuta los algoritmos BFS y BFS paralelo.
    - Mide el tiempo de ejecución y calcula las métricas de rendimiento.
2. **Clase Grafo:**
    - Representa la estructura de datos del grafo mediante un diccionario de listas de adyacencia.
    - Contiene los métodos para:
        - Generar el grafo (`GenerarGrafo`)

- Obtener vecinos (`GetNeighbors`)
- Ejecutar algoritmos de búsqueda (BFS, DFS, BFSParalelo, DFSParalelo)

Esta separación permite mantener una organización clara entre la lógica del sistema y la implementación de los algoritmos.

4.2 Diagrama de componentes/tareas paralelas



4.3 Estrategia de paralelización utilizada

El proyecto implementa dos estrategias principales de paralelización, dependiendo del algoritmo utilizado:

1. Paralelización de BFS (por niveles)

La estrategia utilizada en BFS paralelo es de tipo **level-synchronous**, donde:

- Se procesan todos los nodos de un nivel al mismo tiempo.
- Cada hilo explora los vecinos de un nodo distinto.
- Los resultados se almacenan en una estructura compartida para formar el siguiente nivel.

Esta estrategia es eficiente porque respeta la naturaleza del BFS, garantizando que la primera vez que se alcanza el nodo destino corresponde a la ruta más corta.

2. Paralelización de DFS (por ramas)

En el caso de DFS paralelo, se utiliza una estrategia diferente:

- Se distribuyen los nodos vecinos iniciales entre varios hilos.
- Cada hilo explora una rama del grafo de forma independiente.
- La ejecución se detiene cuando uno de los hilos encuentra el destino.

Esta estrategia permite explorar múltiples caminos simultáneamente, aunque no garantiza encontrar la ruta más corta.

4.4 Herramientas y tecnologías empleadas (C#, TPL, etc.)

El desarrollo del proyecto se realizó utilizando las siguientes herramientas y tecnologías:

- **Lenguaje de programación:** C#
- **Framework:** .NET
- **Biblioteca de paralelismo:** Task Parallel Library (TPL)
- **Estructuras concurrentes:**
  - `ConcurrentDictionary`
  - `ConcurrentBag`
- **Medición de rendimiento:** clase `Stopwatch`

El uso de TPL permite abstraer la complejidad del manejo manual de hilos, facilitando la implementación de soluciones paralelas eficientes y escalables.

5. Implementación Técnica

5.1 Descripción de la estructura del proyecto

El proyecto está desarrollado en C# y organizado en dos componentes principales:

- **Clase `Program`:** contiene el método `Main`, encargado de la ejecución del sistema, interacción con el usuario y medición del rendimiento.
- **Clase `Grafo`:** encapsula la lógica del grafo y la implementación de los algoritmos de búsqueda, tanto en su versión secuencial como paralela.

El grafo se representa mediante un `Dictionary<int, List<int>>`, donde cada clave representa un nodo y su valor es la lista de nodos adyacentes (vecinos).

5.2 Explicación del código clave

1. Generación del grafo

El método `GenerarGrafo(int numNodos, int conexionesPorNodo)` crea un grafo no dirigido de forma aleatoria.

- Se generan conexiones entre nodos utilizando valores aleatorios.
- Se asegura que no haya auto-conexiones (un nodo consigo mismo).
- Cada conexión se agrega en ambos sentidos (grafo no dirigido).

Esto permite simular una red de transporte donde cada nodo representa una ubicación y cada arista una posible ruta.

2. BFS Secuencial

El algoritmo BFS secuencial utiliza:

- Una **cola (`Queue`)** para explorar el grafo por niveles.
- Un **conjunto de visitados (`HashSet`)** para evitar repetir nodos.
- Una lista que representa el camino recorrido.

El algoritmo garantiza encontrar la ruta más corta en términos de número de pasos, lo cual es clave en aplicaciones como navegación.

3. BFS Paralelo

El método `BFSParalelo` es uno de los elementos centrales del proyecto.

Características principales:

- Utiliza `Parallel.ForEach` para procesar los nodos de un mismo nivel en paralelo.
- Controla el número de hilos mediante `MaxDegreeOfParallelism`.
- Emplea estructuras concurrentes:
    - `ConcurrentDictionary` para registrar nodos visitados y sus padres.
    - `ConcurrentBag` para construir el siguiente nivel de nodos.

Flujo del algoritmo:

1. Se inicia con un nodo origen.
2. Se procesan todos los nodos del nivel actual en paralelo.
3. Cada hilo explora los vecinos de su nodo asignado.
4. Los nuevos nodos descubiertos se agregan al siguiente nivel.
5. El proceso se repite hasta encontrar el destino.

Una vez encontrado el destino, se reconstruye el camino utilizando la relación de padres almacenada.

4. DFS Secuencial y Paralelo

- **DFS Secuencial:** utiliza una pila (`Stack`) para explorar el grafo en profundidad.
- **DFS Paralelo:**
  - Divide el trabajo entre hilos desde los vecinos iniciales.
  - Cada hilo explora una rama distinta del grafo.
  - Se utiliza un `ConcurrentBag` para almacenar el resultado encontrado.

Aunque DFS no garantiza la ruta más corta, permite explorar múltiples caminos simultáneamente en su versión paralela.

5.3 Uso de mecanismos de sincronización

El proyecto hace uso de sincronización implícita mediante estructuras concurrentes, evitando el uso manual de bloqueos (`lock`).

Principales mecanismos utilizados:

- **`ConcurrentDictionary`:**
  - Permite insertar nodos de forma segura desde múltiples hilos.
  - Evita condiciones de carrera al verificar si un nodo ya fue visitado.
- **`ConcurrentBag`:**
  - Permite almacenar resultados intermedios sin conflictos entre hilos.
- **`Parallel.ForEach`:**
  - Gestiona automáticamente la distribución de tareas entre hilos.
  - Incluye control de ejecución mediante `state.Stop()` para finalizar anticipadamente cuando se encuentra el destino.

Este enfoque reduce la complejidad del manejo manual de hilos y mejora la seguridad en la ejecución concurrente.

5.4 Justificación técnica de las decisiones tomadas

Las decisiones de diseño e implementación se basan en la necesidad de equilibrio entre rendimiento, simplicidad y seguridad:

- Se utilizó **TPL** en lugar de hilos manuales para simplificar la implementación del paralelismo.
- Se eligieron **estructuras concurrentes** para evitar errores comunes como condiciones de carrera.
- Se implementó **BFS por niveles** para mantener la propiedad de encontrar rutas óptimas.
- Se permitió configurar el número de hilos para analizar la escalabilidad del sistema.
- Se incorporaron métricas de rendimiento para validar el impacto del paralelismo.

Estas decisiones permiten que el sistema sea no solo funcional, sino también escalable, seguro y alineado con prácticas modernas de desarrollo concurrente.

6. Evaluación de Desempeño

6.1 Comparativa entre ejecución secuencial y paralela

Para evaluar el rendimiento del sistema, se realizaron pruebas comparando la ejecución del algoritmo BFS en su versión secuencial y paralela. Las mediciones se realizaron utilizando la clase `Stopwatch`, lo que permitió obtener tiempos de ejecución en milisegundos.

Los resultados muestran que, en algunos casos, la versión paralela logra reducir el tiempo de ejecución respecto a la versión secuencial. Sin embargo, también se observaron escenarios donde la ejecución paralela presenta un rendimiento inferior.

Esto se debe a que el paralelismo introduce costos adicionales, como la creación y gestión de hilos, la sincronización de datos compartidos y el acceso a estructuras concurrentes.

6.2 Métricas: tiempo de ejecución, eficiencia, escalabilidad

Para analizar el comportamiento del sistema, se utilizaron las siguientes métricas:

- **Tiempo de ejecución:** mide el tiempo total que tarda el algoritmo en encontrar la ruta.
- **Speedup:** representa la mejora del tiempo paralelo respecto al secuencial.

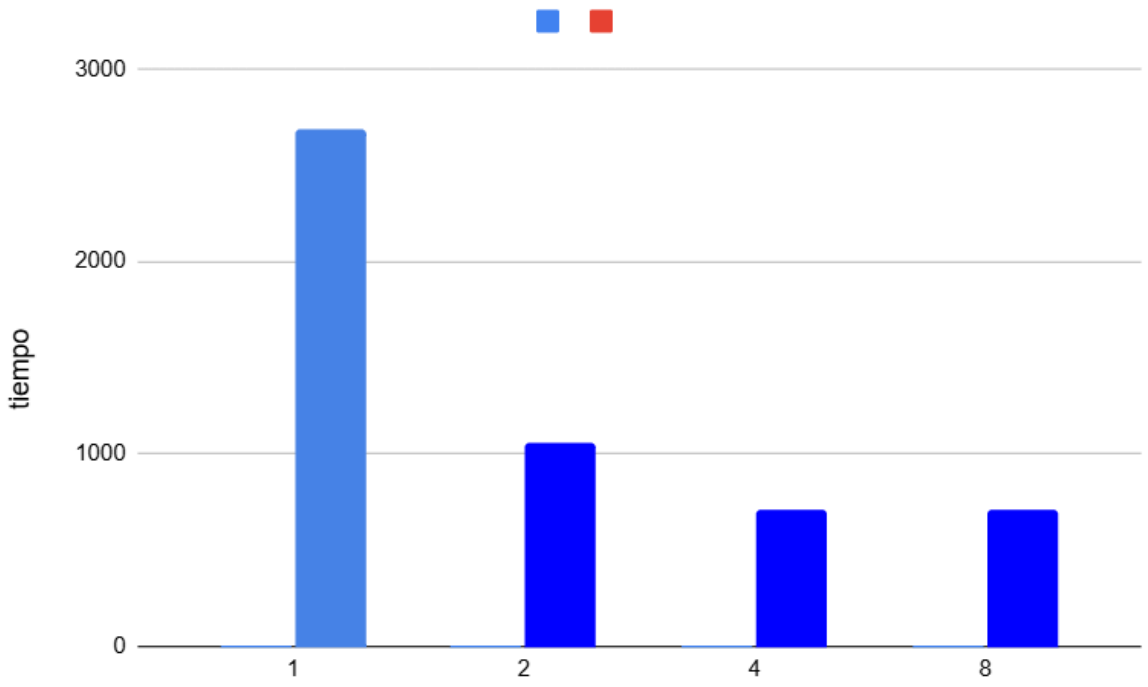
  Speedup = Tiempo Secuencial / Tiempo Paralelo

- **Eficiencia:** indica qué tan bien se aprovechan los núcleos del procesador.

  Eficiencia = Speedup / Número de núcleos

Estas métricas permiten evaluar no solo si el sistema es más rápido, sino también si el uso de múltiples núcleos es eficiente.

6.3 Gráficas o tablas con resultados



tiempo vs núcleos usados

Num_Nodos	Destino	Nucleos	Tiempo_Secuencial_ms	Tiempo_Paralelo_ms	Speedup	Eficiencia
1,000,000	74	1	2557	2684	0.953	95.27%
1,000,000	74	2	2557	1062	2.408	120.39%
1,000,000	74	4	2557	710	3.601	90.04%
1,000,000	74	8	2557	713	3.586	44.83%
10,000,000	4987	1	3489	3669	0.951	95.09%
10,000,000	4987	2	3489	2339	1.492	74.58%
10,000,000	4987	4	3489	1933	1.805	45.12%
10,000,000	4987	8	3489	2446	1.426	17.83%
10,000,000	4987	8	31968	8796	3.634	45.43%
475,400	7844	8	337	188	1.793	22.41%
1,000,000	477	8	1357	469	2.893	36.17%

6.4 Análisis de cuellos de botella o limitaciones

Durante la ejecución del sistema se identificaron varias limitaciones que afectan el rendimiento:

1. Overhead del paralelismo

El uso de múltiples hilos implica costos adicionales como:

- Creación y gestión de tareas
- Sincronización entre hilos
- Coordinación de estructuras compartidas

En grafos pequeños o medianos, este costo puede ser mayor que el beneficio del paralelismo, haciendo que la versión paralela sea más lenta que la secuencial.

2. Acceso a memoria compartida

El uso de estructuras como `ConcurrentDictionary` y `ConcurrentBag` introduce un costo adicional debido a la sincronización interna que estas requieren para evitar conflictos entre hilos.

Esto puede generar cuellos de botella cuando múltiples hilos intentan acceder o modificar datos al mismo tiempo.

3. Desbalance de carga
No todos los nodos generan la misma cantidad de trabajo. Algunos pueden tener más vecinos que otros, lo que provoca que ciertos hilos trabajen más que otros, reduciendo la eficiencia global del sistema.

4. Tamaño del grafo
El paralelismo muestra mejores resultados en grafos grandes, donde el volumen de trabajo justifica el uso de múltiples hilos.

En grafos pequeños, la versión secuencial suele ser más eficiente debido a la ausencia de overhead.

7. Trabajo en Equipo

7.1 Descripción del reparto

de tareas todo lo hice yo yaque trabaja solo

7.2 Herramientas utilizadas para coordinación (Git, AzureDevops, Trello, Excel, etc.)

8. Conclusiones

8.1 Principales aprendizajes técnicos

Este proyecto permitió entender de forma práctica cómo funciona la programación paralela aplicada a problemas reales como la búsqueda de rutas en grafos. Se logró implementar versiones secuenciales y paralelas de BFS y DFS, y comparar su rendimiento.

También se aprendió a utilizar herramientas como la **Task Parallel Library (TPL)** y estructuras concurrentes en C#, entendiendo su importancia para manejar múltiples hilos de forma segura.

8.2 Retos enfrentados y superados

Uno de los principales retos fue que el algoritmo paralelo no siempre era más rápido que el secuencial. Esto llevó a analizar aspectos como el overhead del paralelismo, la sincronización y el uso de memoria compartida.
Además, fue necesario definir correctamente cómo dividir el trabajo entre hilos, especialmente en los algoritmos BFS y DFS.

8.3 Posibles mejoras o líneas futuras

Como mejora futura, se podría trabajar con datos reales en lugar de grafos aleatorios, implementar algoritmos más avanzados como Dijkstra o A*, y optimizar el uso del paralelismo para obtener mejores resultados.

9. Referencias

Fuentes bibliográficas, técnicas o académicas consultadas

- Microsoft. (2023). *Task Parallel Library (TPL)*.

  https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-parallel-library-tpl
- Microsoft. (2023). *Parallel.ForEach Method*. https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.parallel.foreach
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing* (2nd ed.). Pearson.
- Wikipedia. (2024). *Breadth-First Search*. https://en.wikipedia.org/wiki/Breadth-first_search
- Wikipedia. (2024). *Depth-First Search*. https://en.wikipedia.org/wiki/Depth-first_search

10. Anexos

10.1 Manual de ejecución del sistema

Para ejecutar el sistema, se deben seguir los siguientes pasos:

1. Abrir el proyecto en un entorno de desarrollo compatible con C#, como Visual Studio.
2. Compilar el proyecto para asegurar que no existan errores.
3. Ejecutar el programa desde la consola.

Al iniciar, el sistema solicitará:

- La cantidad de nodos del grafo.
- El nodo destino a buscar.

Una vez ingresados los datos, el sistema:

- Genera el grafo automáticamente.
- Ejecuta el algoritmo BFS en su versión secuencial y paralela.
- Muestra los tiempos de ejecución.
- Calcula métricas como speedup y eficiencia.
- Realiza pruebas de escalabilidad con diferentes cantidades de núcleos.

Finalmente, se muestran las rutas encontradas y los resultados de rendimiento.

10.2 Capturas adicionales, pruebas complementarias

```
Consola de depuración de Microsoft Visual Studio
Núcleos lógicos disponibles: 8
¿Cuántos nodos quieres? 100000
¿Cuál será el destino? 47
Generando grafo con 100000 nodos...
BFS Secuencial: 255 ms
BFS Paralelo (8 núcleos): 91 ms

Resultado BFS Secuencial:
1 43861 39987 42835 80390 4572 69805 87978 89626 4931 47

Resultado BFS Paralelo:
1 43861 39987 42835 80390 4572 69805 87978 89626 4931 47

--- MÉTRICAS BFS (máximo paralelismo) ---
Speedup: 2.80
Eficiencia: 0.35
```

```
============================
PRUEBAS DE ESCALABILIDAD BFS
============================

Núcleos: 1
Tiempo: 80 ms
Speedup: 3.19
Eficiencia: 3.19

Núcleos: 2
Tiempo: 74 ms
Speedup: 3.45
Eficiencia: 1.72

Núcleos: 4
Tiempo: 63 ms
Speedup: 4.05
Eficiencia: 1.01

Núcleos: 8
Tiempo: 57 ms
Speedup: 4.47
Eficiencia: 0.56

Fin de las pruebas.
```

Enlace al repositorio de Git (público)

https://github.com/Eduardo04alv/paralela-proyecto-final/tree/main